

Trabalho Prático — Gerência de Configuração

Tema: Docker Compose

Visão geral

- **Equipe:** 5 a 7 integrantes
- **Objetivo:** pesquisa aplicada sobre Docker Compose + protótipo funcional que demonstre os conceitos na prática
- **Peso:** até 10,0 pontos (2ª unidade)
- **Formato:** repositório GitHub (público) + documentação em Markdown + apresentação oral
- **Regra da turma:** não há repetição de tema entre grupos
- **Prazo:** a confirmar com o professor

Entregáveis obrigatórios

1. Repositório GitHub (público)

- [] README com nome da UFCA, disciplina, nomes completos da equipe e professor
- [] Documentação técnica majoritariamente em Markdown, organizada e com hyperlinks
- [] Código-fonte de todos os exemplos práticos (arquivos `docker-compose.yml`, `Dockerfiles` etc.)
- [] Instruções completas de instalação, configuração e execução (como rodar o ambiente do zero)
- [] Seção de referências bibliográficas
- [] Histórico de commits distribuído ao longo do tempo (commits regulares — não vale um único commit no fim)

2. Conteúdo obrigatório da documentação

- [] Pesquisa bibliográfica sobre Docker Compose
- [] Contextualização: por que o Compose surgiu e que problema resolve (orquestrar múltiplos containers sem depender de vários comandos `docker run` manuais)
- [] Conceitos e mecanismos: `services`, `networks`, `volumes`, variáveis de ambiente, dependência entre serviços (`depends_on`), `healthchecks`
- [] Comparação prática: Compose vs. containers gerenciados manualmente — vantagens, limitações, cenários de uso
- [] Análise crítica: maturidade, ecossistema, limitações (não é um orquestrador de produção como Kubernetes), perspectivas de adoção

3. Apresentação oral

- [] Todos os integrantes presentes, participando de forma equilibrada
- [] Cada integrante precisa dominar o conteúdo completo — a nota individual depende disso, inclusive nas perguntas
- [] Duração a confirmar com o professor (referência do tema de Rust na mesma turma: 12–15 min + até 5 min de perguntas)

Exemplos sugeridos para a demonstração prática

- Aplicação multi-serviço orquestrada via `docker-compose.yml` (ex.: API + banco de dados + cache)
- Comunicação entre containers pela rede interna do Compose (service discovery pelo nome do serviço, sem IP fixo)
- Persistência de dados com volumes (dados sobrevivem a `docker compose down/up`)
- Ordem de inicialização correta com `depends_on` + `healthcheck` (evitar a aplicação subir antes do banco estar pronto)
- Comparação lado a lado: subir os mesmos serviços via `docker run` manual (múltiplos comandos, rede criada na mão) vs. `docker compose up` (um único comando, tudo declarado em arquivo)

Ponto para a análise crítica

O Docker Compose foi desenhado para orquestração local / desenvolvimento — não é um substituto para um orquestrador de produção como Kubernetes (sem auto-scaling, sem failover multi-host, sem service mesh nativo). Bom contraponto para discutir limites de escala e o momento de migrar de Compose para uma solução de orquestração mais robusta.

Divisão de trabalho sugerida (5–7 pessoas)

- **Contexto e fundamentos** (1–2 pessoas) — pesquisa bibliográfica, histórico, conceitos de services/networks/volumes
- **Exemplos práticos** (2–3 pessoas) — ambiente multi-serviço, `depends_on/healthcheck`, demonstração comparativa com `docker run`
- **Análise crítica e ecossistema** (1–2 pessoas) — maturidade, limitações frente ao Kubernetes, cenários de adoção
- **Documentação e repositório** (1 pessoa, com apoio de todos) — README, organização do Markdown, instruções, referências

Recomendado: revisão cruzada entre frentes e ensaio da apresentação completa por todos (não só da própria parte).

Observação: peso, unidade e regra de não repetição de tema seguem o edital da disciplina. Prazo e duração exata da apresentação não estavam disponíveis nesta sessão — confirme com o professor.